

A few ways to achieve this are:

- PREEMPT_RT
- RTLinux
- Xenomai
- RTAI

PREEMPT_RT: Originally referred to as the sleeping spinlocks patch, it was developed by Ingo Molnar and Thomas Gleixner for the 2.6.22 kernel. It provides full kernel pre-emption by replacing spinlocks with mutexes.

RTLinux: This uses a micro-kernel approach that runs the entire Linux kernel as a fully pre-emptive process. RTLinux was developed in FSM Labs, which was later acquired by Wind River. Therefore, currently there are two versions of RTLinux — a proprietary version offered by Wind River and an open source version that is available under GPL.

Xenomai: This uses a dual-kernel method, where non-real-time tasks are run on the Linux kernel and real-time tasks are run on the Xenomai kernel. The two kernels communicate via a virtual interrupt controller. Xenomai was meant to provide real-time support to user space applications on a Linux based platform.

RTAI: This is a community developed extension to the Linux kernel, which lets users write applications that can meet strict real-time deadlines. It consists of a patch that introduces a hardware abstraction layer. It also exposes a set of services, which makes programming on RTAI easier.

An experimental setup

Some of the features of PREEMPT-RT are sleeping spinlocks, threaded interrupts, high resolution timers and priority inheritance. We chose the PREEMPT_RT patch and applied it to the vanilla Linux kernel for our experiments. The only necessary configuration for a real-time Linux kernel is selecting 'Fully Pre-emptible Kernel' in the *menu config* options.

The test environment: All the tests were performed on two virtual machines (vanilla and real-time patched) created on VirtualBox. The tests were conducted on two different

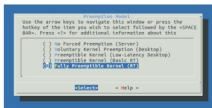


Figure 1: Kernel configuration options using *menu config*

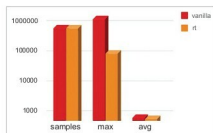


Figure 2: Latency results under no-load conditions

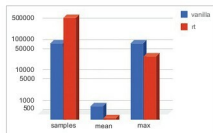


Figure 3: Latency results with CPU-intensive load in user mode

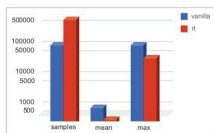


Figure 4: Latency results with CPU-intensive load in kernel mode

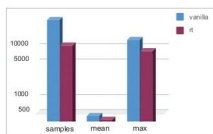


Figure 5: Latency results with memory-intensive load

kernel versions and patches too. The details are as mentioned below.

Linux distribution: Ubuntu 16

Kernel version: 4.9.40 and 4.11.12

PREEMPT_RT patch version: RT30

and RT14

Cores: 1- 4, depending on the tests

RAM: 2GB

Scheduling latency: Scheduling

latency is the time taken by the kernel to schedule a task. It is measured as the difference between the time a task wakes up and the time it is run.

The task execution time depends on scheduling latency. An RTOS needs to be deterministic and predictable and, therefore, scheduling latency is of paramount importance. Hence, this is chosen as our parameter of interest.

Latency test – Cyclicttest: Cyclicttest is a part of RT-tests and is used to measure scheduling latency. It starts running a non-real-time master thread, which starts a defined number of threads with a real-time priority. These threads are woken up periodically by an expiring timer. The difference between the programmed and effective wake-up times is calculated. In this manner, minimum, average and maximum latency values (in microseconds) are calculated and printed.

Loads:

1. CPU-intensive applications
2. Memory-intensive applications
3. Hackbench
4. KCompile

Implementation and results

Latency under no-load conditions:

Cyclicttest was run on both kernels with no external load (Figure 2).

Latency with a CPU-intensive load: A simple infinite *for* loop was used as a user mode CPU-intensive load, and the same configuration with an additional system call to get the process ID was used as the kernel mode load. Cyclicttest was run under these conditions (Figures 3 and 4).

Latency with a memory-intensive load:

The load, in this case, is created using the *dd* command to create a 500MB file of zeros.

Latency with Hackbench as load: Hackbench is a stress test on the Linux scheduler. It creates a certain number of pairs of threads, which communicate through pipes/